# MACHINE LEARNING PROJECT

# Prediction of the Churn Rate of a Mobile Company

Daniel Badaire / Manuel Griseri

Master 2 : Quantitative Finance

Academic year 2025/2026

# 1 Introduction

The goal of this report is to find a model that can accurately predict customer churn for a mobile phone operator and to evaluate the fairness of our model with respect to the sensitive variable gender. The first part is therefore dedicated to explaining the models used and to assessing and comparing them through appropriate metrics in order to identify the best one. Subsequently, we will study the independence, separation, and sufficiency of the optimal classifier found in the first part.

# 2 Exploratory data analysis

We will work with a dataset that contains the binary variable we want to predict (*Churn*) and 10 covariates: five continuous quantitative variables (*CreditScore*, *Age*, *Tenure*, *Balance*, *Salary*), one discrete quantitative variable (*NumOfProducts*), and four categorical variables (*Geography*, *Gender*, *HasCrCard*, *IsActiveMember*). The presence of missing values was first checked, and none were found. The figures below show the boxplots of the continuous variables, separated between individuals who churned and those who did not, while for the discrete and categorical variables the corresponding confusion matrices are reported. This provides insight into the distribution of the covariates as a function of churn, and consequently which variables may have the strongest influence on the target variable. For example, from the confusion matrices we can observe that being an active member and the number of products purchased show a strong association with churn, whereas gender provides relatively little discriminative information.

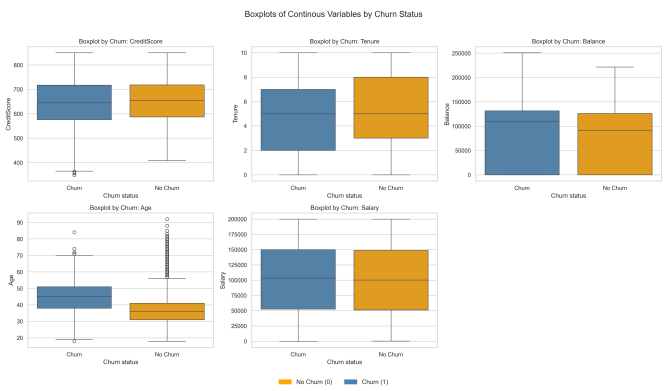


Figure 1: Boxplots of continous variables

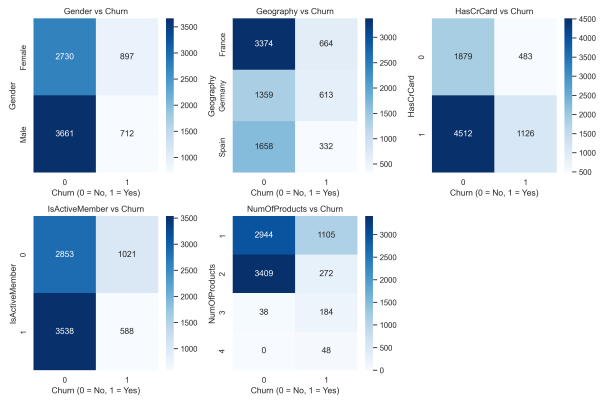


Figure 2: Confusion matrixes of dicrete and categorical variables

Next, the Pearson correlation among the variables was examined, and it resulted globally weak. Therefore, there is no evidence of linear correlation among the variables, although intuition may suggest that some relationships could exist (e.g., between salary and credit score). These relationships may be non-linear, in which case more advanced techniques such as copulas would be needed to detect them. Finally, to assess the validity of the normality assumption for the continuous features conditioned on churn, Jarque-Bera tests were performed. However, only the distribution of *Age* conditioned on the individual having churned showed indications of normality. It is also important to note that there is a significant class imbalance, as individuals who churn represent only about 20% of the dataset. Therefore, it will be necessary to adopt appropriate techniques to handle this issue, since some models are sensitive to this characteristic.

# 3 Supervised learning models

In this third chapter, we briefly introduce the various machine learning models used to predict the churn rate. For each model, we describe the underlying assumptions, its parameters and hyperparameters,

its advantages and disadvantages, and the final mathematical formulation of the classifier. Let $X_{nj}$, for $n = 1, \ldots, N$ and $j = 1, \ldots, J$, be a random variable representing the $j$-th feature of individual $n$, while $Y_n$ is a binary random variable taking values in $\{0, 1\}$ and indicating whether individual $n$ has churned.

### 3.1 Logistic classifier

The logistic classifier assumes a linear relationship between the log-odds of $Y_n$ and the features $(X_{nj})$, with independent observations and a binary response $Y_n$. Its parameters are the weights $w_j$ and the intercept $b$, while typical hyperparameters involve the choice of L1/L2 regularization and its strength $\lambda$. It is simple, interpretable and efficient, but limited to linear decision boundaries and can suffer from multicollinearity. For each individual $n$, the predicted label is

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \left[ \sigma\left(b + \sum_{j=1}^{J} w_j X_{nj}\right) \mathbb{I}(y = 1) + \left(1 - \sigma\left(b + \sum_{j=1}^{J} w_j X_{nj}\right)\right) \mathbb{I}(y = 0) \right] \tag{1}$$

### 3.2 Gaussian Naive Bayes

The Gaussian Naive Bayes classifier assumes conditional independence of the features $(X_{nj})$ given $Y_n$, with each feature modeled by a Gaussian per class. It uses class priors and feature means and variances as parameters, has almost no hyperparameters, is fast and stable on small datasets, but may perform poorly when features are strongly correlated. For each individual $n$, the predicted label is

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \left\{ \log \mathbb{P}(Y_n = y) - \frac{1}{2} \sum_{j=1}^{J} \left[ \log(2\pi\sigma_{j,y}^2) + \frac{(X_{nj} - \mu_{j,y})^2}{\sigma_{j,y}^2} \right] \right\} \tag{2}$$

### 3.3 K-Nearest-Neighbors

The $k$-Nearest Neighbors classifier assigns each individual $n$ the most frequent label among the $k$ closest observations in the training set, where $\mathcal{N}_k(n)$ denotes the set of its $k$ nearest neighbors. It has no model parameters, while its main hyperparameters are $k$ and the chosen distance metric; it is simple and flexible but can be slow on large datasets and sensitive to feature scaling. For each individual $n$, the predicted label is

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \sum_{i \in \mathcal{N}_k(n)} \mathbb{I}(Y_i = y) \tag{3}$$

### 3.4 Support Vector Machine

The Support Vector Machine classifier seeks a decision boundary that maximizes the margin between the two classes, using a kernel function to allow nonlinear separation when needed. Its parameters are the normal vector $w$ and intercept $b$, while its main hyperparameters are the regularization parameter $C$ and the kernel choice. It is effective in high-dimensional spaces but can be sensitive to the choice of $C$ and kernel parameters. For each individual $n$, the predicted label is

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \mathbb{I}\Big(y = \mathbb{I}\Big(w^\top X_n + b \geq 0\Big)\Big) \tag{4}$$

### 3.5 Random forest

The Random Forest classifier aggregates the predictions of many decision trees built on bootstrap samples, each using random subsets of features at every split. It has no explicit model parameters, while its main hyperparameters are the number of trees, the maximum tree depth, and the number of features sampled at each split. It is robust, handles nonlinearities well, and reduces variance, but can be less

interpretable and slower when many trees are grown. For each individual $n$, the predicted label is ($T_m$ denotes the $m$-th tree in the forest)

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \sum_{m=1}^{M} \mathbb{I}(T_m(X_n) = y) \tag{5}$$

### 3.6 AdaBoost

The AdaBoost classifier builds an ensemble of weak learners by iteratively reweighting the training observations so that misclassified points receive higher weight in later rounds. Its parameters are the weights $\alpha_m$ assigned to each weak learner, while its main hyperparameters are the number of boosting rounds and the learning rate. It is powerful and often achieves low bias, but can be sensitive to noise and outliers. For each individual $n$, the predicted label is

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \sum_{m=1}^{M} \alpha_m \, \mathbb{I}(T_m(X_n) = y) \tag{6}$$

### 3.7 XGBoost

XGBoost is a gradient boosting method that builds regression trees sequentially, each fitted to the negative gradients of a regularized loss. Its parameters are the leaf values of all boosted trees, while its main hyperparameters include the learning rate, the number of boosting rounds, the maximum depth, and regularization terms. It is powerful and flexible but can be sensitive to hyperparameter tuning. For each individual $n$, the predicted label is (each $f_m$ is a regression tree contributing to the boosted score)

$$\hat{Y}_n = \arg\max_{y \in \{0,1\}} \left[ \sigma\left( \sum_{m=1}^{M} f_m(X_n) \right) \mathbb{I}(y = 1) + \left( 1 - \sigma\left( \sum_{m=1}^{M} f_m(X_n) \right) \right) \mathbb{I}(y = 0) \right] \tag{7}$$

## 4 Model performance

In order to evaluate and compare the performance of the classifiers introduced above, the dataset was split into a training set (80%), used to fit the models, and a test set (20%), used to compute the final metrics. In this chapter, we first define the metrics employed and then present the obtained results.

### 4.1 Metrics employed

Firstly, the AUC was computed, defined as the area under the ROC curve, where the ROC curve plots the True Positive Rate against the False Positive Rate at varying classification thresholds; values closer to 1 (optimal ROC) indicate better performance. Moreover, calling TP the number of true positives, FP false positives, TN true negatives and FN false negatives, the following metrics were used (again, values closer to 1 indicate better performance) :

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP} \tag{8}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad \text{F1} = \frac{2\,\text{Precision}\,\text{Recall}}{\text{Precision} + \text{Recall}} \tag{9}$$

In general, AUC is the most important metric, as it provides a global indication of the overall quality of a model. In our case, recall is also particularly important, since a low recall implies a high number of false negatives (i.e., individuals who churn but are classified as non-churners), which is a serious problem for a company because it results in losing customers. There is therefore a clear trade-off between precision and recall, which motivates the use of the F1 score, as it indicates whether there is a good balance between these two metrics.

### 4.2 Performance comparison

The class imbalance (only 20% churners) may compromise the predictive ability of some models, and in particular it tends to lower recall simply because being a churner is less probable in the full dataset. For this reason, the *balanced class weights* method was applied to the Logistic Regression, SVM, and Random Forest models (although other methods such as SMOTE also exist). For XGBoost, instead, the *scale position weight* ratio method was used.

Now, cross-validation was carried out by repeatedly splitting the training set into different training and validation subsets, in order to obtain multiple values of the previously discussed metrics for all the models under consideration. This allowed us to plot the empirical distribution of each metric for each model on both the training and validation sets, which we can compare using violin plots. Figure 3 shows such a multiple violin plot for the recall metric. Several observations can be made from this plot: the greater variability of the validation distribution compared to the training one, the clear overfitting of the Random Forest, and the excellent performance of XGBoost and SVM contrasted with the poor performance of GNB and KNN.

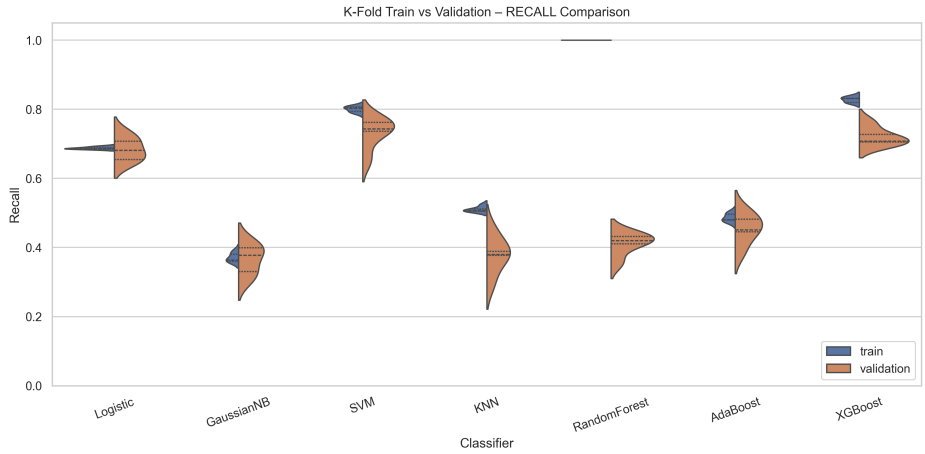


Figure 3: Violin plots for the recall

The final performance, however, must be evaluated on the test set. For this reason, Table 1 reports, for each model, the value of the various metrics computed on the test set. We can observe that the best model according to the AUC is XGBoost, which also achieves a good recall, the two metrics that matter most for our purposes. It is also useful to note that, before handling class imbalance, the model had a much higher precision (0.805) but a much lower recall (0.512). The last column of the table reports the test AUC values for each model after appropriate hyperparameter tuning using the *Randomized Search* method (although more advanced hyperparameter optimization methods exist, such as Bayesian Optimization, we chose to rely on simpler approaches. This allowed us to tune our models more quickly while keeping the procedure computationally efficient). We can observe an improvement in the performance of all models and a confirmation that XGBoost is the best-performing model.

Table 1: Final Test Metrics for All Models

| **Model** | **Accuracy** | **F1** | **Precision** | **Recall** | **AUC** | **AUC (After Tuning)** |
|---|---|---|---|---|---|---|
| Logistic | 0.726 | 0.507 | 0.397 | 0.702 | 0.777 | 0.777 |
| GaussianNB | 0.821 | 0.451 | 0.587 | 0.366 | 0.785 | — |
| SVM | 0.789 | 0.587 | 0.485 | 0.742 | 0.847 | 0.848 |
| KNN | 0.838 | 0.497 | 0.663 | 0.398 | 0.779 | 0.841 |
| Random Forest | 0.868 | 0.581 | 0.799 | 0.457 | 0.844 | 0.855 |
| AdaBoost | 0.862 | 0.576 | 0.754 | 0.466 | 0.851 | 0.853 |
| XGBoost | 0.806 | 0.606 | 0.513 | 0.739 | 0.869 | 0.870 |

As a graphical confirmation, Figure 4 shows the dominance of the XGBoost ROC curve over the ROC curves of the other models.

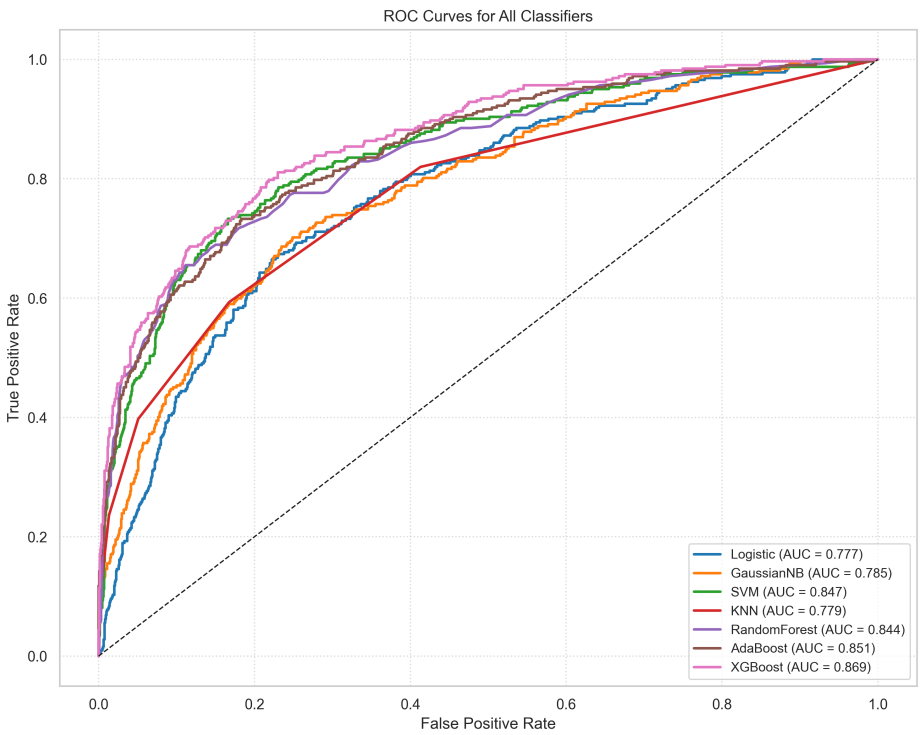


Figure 4: ROC and AUC for each model

# 5 Fairness analysis with respect to gender

The model we would recommend is therefore XGBoost, and it is on this model that the fairness analysis with respect to the gender variable will be conducted in this chapter. Algorithmic fairness is evaluated through three classical families of criteria in machine learning: *Independence*, *Separation*, and *Sufficiency*. These notions are expressed in terms of conditional dependencies between the prediction $\hat{Y}$, the true label $Y$, and the sensitive attribute $S$ (here: gender).

$$\text{Independence: } \hat{Y} \perp S, \qquad \text{Separation: } \hat{Y} \perp S \mid Y, \qquad \text{Sufficiency: } Y \perp S \mid \hat{Y} \tag{10}$$

As shown by [1], these three criteria are mutually incompatible when base rates differ across groups. Any fairness analysis must therefore be interpreted as a structural trade-off between these definitions.

## 5.1 Independence

This criterion requires that:

$$P(\hat{Y} = 1 \mid S = \text{Female}) \approx P(\hat{Y} = 1 \mid S = \text{Male}) \tag{11}$$

The positive prediction rates are: 0.3724 for female and 0.2318 for male. Absolute gap:

$$|DP_{\text{gap}}| = 0.1406 < 0.05 \tag{12}$$

Demographic Parity is almost fully satisfied, since the discrepancy remains below the commonly accepted 5% threshold.

### 5.2 Separation

Separation requires comparable predictive behaviour in each group:

$$TPR_g = P(\hat{Y} = 1 \mid Y = 1, S = g), \qquad FPR_g = P(\hat{Y} = 1 \mid Y = 0, S = g) \tag{13}$$

| Group | TPR | FPR |
|---|---|---|
| Female | 0.7433 | 0.2454 |
| Male | 0.7482 | 0.1366 |

$$|TPR_{\text{gap}}| = 0.0048 < 0.05, \qquad |FPR_{\text{gap}}| = 0.1088 > 0.05 \tag{14}$$

While TPR remains balanced across genders, the FPR gap exceeds the acceptable threshold. Therefore, the model does not satisfy Separation, as error rates are not consistently balanced between groups.

### 5.3 Sufficiency

Sufficiency requires:

$$P(Y = 1 \mid \hat{Y}, S = \text{Female}) \approx P(Y = 1 \mid \hat{Y}, S = \text{Male}) \tag{15}$$

Predictive Parity (PPV) are: 0.5092 for female and 0.5025 for male.

$$|PPV_{\text{gap}}| = 0.0067 < 0.05 \tag{16}$$

The model satisfies Sufficiency, meaning that predicted risk scores have the same meaning across gender groups. In other words, for a given predicted score, the actual probability of churn is consistent between men and women. Also, calibration results are globally consistent across genders: for each score bin, the observed churn rates follow the same increasing trend and remain close in magnitude.

## 6 Conclusion

From the first part, XGBoost emerges as the most suitable model. It achieves the strongest recall and overall balance, making it highly effective at identifying at-risk customers while maintaining solid global performance (AUC). SVM is a strong alternative. RandomForest and AdaBoost provide good precision but miss too many subtle churners, while GaussianNB and KNN perform insufficiently for this task. Strategically, a model prioritizing recall is essential for telecom retention efforts, where keeping a customer is far cheaper than acquiring one and where small incentives can prevent costly churn. XGBoost best supports a proactive, data-driven retention strategy.

From the second part, this model demonstrates overall fair behaviour with respect to gender. It satisfies Independence and Sufficiency, indicating that predictions and predicted risk levels are interpreted consistently across groups. Although the TPR component of Separation is balanced, the FPR gap exceeds the acceptable threshold, meaning full error-rate parity is not achieved. Nevertheless, no evidence of harmful or systematic discrimination emerges, and the model can be considered fair within the scope of this study.

## References


[1] Jon Kleinberg, Sendhil Mullainathan, and Manish Raghavan. Inherent trade-offs in the fair determination of risk scores. *ITCS*, 2016.

[2] Mathilde Mougeot. Lecture notes, slides and code on machine learning. ENSIIE, 2025.

[3] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction.* MIT Press, 2022.

[4] Vincent Runge. Classification supervisée. Course handout, Université d'Évry, 2025.

# Appendix

## A Anomaly detection

In this first appendix, we apply two anomaly detection methods, namely Isolation Forest and Local Outlier Factor. Observations in the training set whose anomaly scores exceed the 97th percentile for both methods are removed. The predictive models are then retrained on the filtered training data, and their test performance is re-evaluated.

### A.1 Isolation forest

Isolation Forest is an unsupervised anomaly detection algorithm based on the idea that anomalies are easier to isolate than normal observations. It recursively partitions the feature space using randomly generated binary trees. At each node, a feature is selected uniformly at random and split at a random value within its range. As a result, anomalous observations tend to be isolated after fewer splits, leading to shorter path lengths. For a given observation $x$, the anomaly score is defined as

$$s(x, n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}} \tag{17}$$

where $\mathbb{E}[h(x)]$ is the average path length of $x$ across the ensemble of isolation trees, and $c(n)$ is a normalization constant corresponding to the expected path length in a random binary search tree:

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \tag{18}$$

with $H(\cdot)$ denoting the harmonic number.

### A.2 Local Outlier Factor

Local Outlier Factor (LOF) is an unsupervised anomaly detection method that identifies anomalies by comparing the local density of an observation with that of its neighbors. Unlike global approaches, LOF focuses on the local structure of the data and is therefore able to detect both global and local outliers. For a given observation $x_i$, let $N_k(x_i)$ denote its set of $k$ nearest neighbors. The local reachability density (LRD) of $x_i$ is defined as

$$\text{lrd}_k(x_i) = \left( \frac{1}{|N_k(x_i)|} \sum_{x_j \in N_k(x_i)} \text{reach-dist}_k(x_i, x_j) \right)^{-1} \tag{19}$$

where the reachability distance is given by

$$\text{reach-dist}_k(x_i, x_j) = \max\{k\text{-dist}(x_j),\, d(x_i, x_j)\} \tag{20}$$

The Local Outlier Factor score is then computed as

$$\text{LOF}_k(x_i) = \frac{1}{|N_k(x_i)|} \sum_{x_j \in N_k(x_i)} \frac{\text{lrd}_k(x_j)}{\text{lrd}_k(x_i)} \tag{21}$$

When the LOF value is close to 1, the observation has a density similar to that of its neighbors and can be considered typical. Values smaller than 1 indicate that the observation lies in a denser region than its neighbors and is therefore a strong inlier. Conversely, LOF values greater than 1 reveal observations located in sparser regions, meaning they are less dense than their neighbors and can be interpreted as potential outliers.

### A.3 New performance after outliers removal

Removing the observations identified as outliers from the training set leads to test performances that are overall very similar to those obtained with the full training data. For all classifiers, both AUC and recall exhibit only marginal variations after outlier removal, with no systematic improvement or degradation. This suggests that the detected anomalies do not strongly influence the learned decision boundaries and that the models are relatively robust to the presence of such outliers. Consequently, anomaly filtering in this setting appears to have a limited impact on predictive performance, while primarily acting as a data-cleaning step rather than a performance-enhancing one.

On the other hand, removing outliers inevitably results in a loss of information, as these observations may still carry meaningful patterns related to rare but relevant behaviors. An alternative strategy is therefore to retain these data points while assigning them different weights during training, allowing the models to reduce their influence without discarding potentially informative observations altogether.

Table 2: Model performance before and after anomaly removal

| **Model** | **AUC (Before)** | **Recall (Before)** | **AUC (After)** | **Recall (After)** |
|---|---|---|---|---|
| Logistic | 0.7766 | 0.7019 | 0.7776 | 0.6894 |
| GaussianNB | 0.7850 | 0.3665 | 0.7873 | 0.3727 |
| SVM | 0.8470 | 0.7422 | 0.8463 | 0.7484 |
| KNN | 0.7792 | 0.3975 | 0.7800 | 0.3913 |
| Random Forest | 0.8442 | 0.4565 | 0.8435 | 0.4410 |
| AdaBoost | 0.8507 | 0.4658 | 0.8485 | 0.4689 |
| XGBoost | 0.8692 | 0.7391 | 0.8690 | 0.7298 |

In case of a significant degradation in performance after removing outliers, this would indicate that those observations were actually informative for the prediction task. Rather than being pure noise, such points would carry meaningful signal related to the target variable, and their removal would lead to a loss of relevant information. This situation suggests that the model was effectively exploiting patterns present in these atypical observations and that alternative strategies, such as reweighting or robust modeling, may be more appropriate than outright removal.

Among the considered models, distance-based and margin-based classifiers are the most sensitive to outliers. In particular, KNN is highly affected because anomalous observations can distort local neighborhood structures, while SVM may be influenced by outliers that lie close to or beyond the decision margin. Linear models such as Logistic Regression also exhibit sensitivity, as outliers can disproportionately impact the estimated coefficients. In contrast, tree-based ensemble methods, including Random Forest, AdaBoost, and XGBoost, are generally more robust to outliers due to their reliance on recursive splits rather than global distance or linear assumptions.

# B Principal Component Analysis

Principal Component Analysis (PCA) is a multivariate statistical method that transforms a set of correlated variables into a new set of uncorrelated variables called principal components. These components are ordered by their importance according to the amount of variance they explain. This method is particularly useful for visualizing complex data or simplifying interpretation. We centered and scaled the data before performing PCA in order to harmonize the range and units of the variables. By doing so, we avoid having variables with large amplitudes dominate the analysis and distort the results. We do not include NumProducts because almost all individuals have the value 1, so it contributes very little to the variance.

From Figure 5, we can observe that the variance explained by the five principal components is overall fairly uniform. Therefore, in order to reach the commonly used threshold of 95% explained variance, all five components would need to be retained.

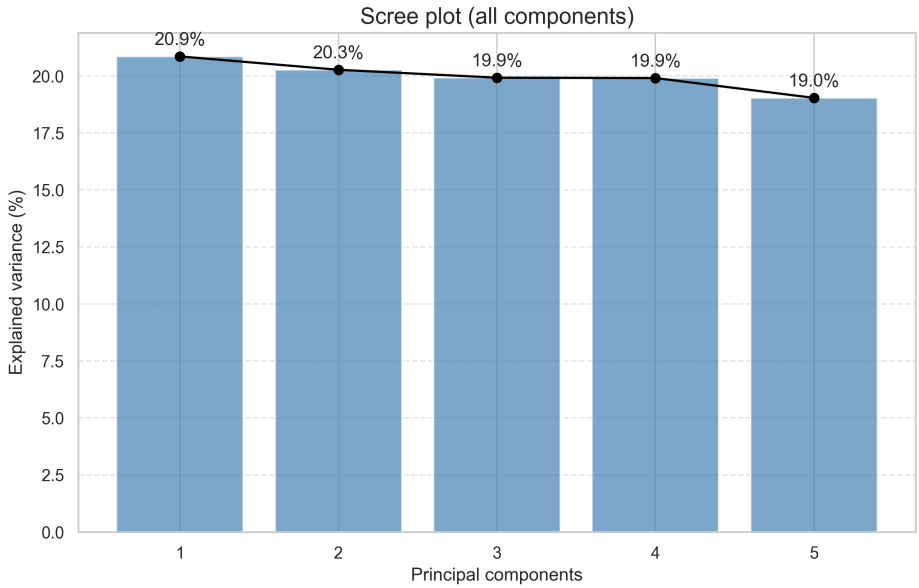


Figure 5: Explained variance of each component

Figure 6 shows the PCA correlation circle for the first two principal components, which together explain approximately 41% of the total variance. The length of the vectors indicates that *Age*, *Balance*, *Salary*, and *CreditScore* are well represented in the PC1-PC2 plane, while *Tenure* is only weakly captured by the first two components. The first principal component (PC1) is mainly driven by *Age* and *Balance*, which are strongly and positively correlated. PC1 can therefore be interpreted as a dimension related to customer seniority and financial exposure. The second principal component (PC2) is dominated by *Salary* with a positive loading and by *CreditScore* with a negative loading, indicating a strong negative correlation between these two variables. PC2 thus contrasts income level with creditworthiness.

Figure 7 shows the projection of the observations onto the first two principal components. Each point represents a customer, colored according to the target variable (remaining with the company versus leaving the company). The two classes largely overlap in the PC1-PC2 space, indicating that the first two principal components alone do not provide a clear linear separation between customers who remain and those who leave. This is consistent with the fact that PC1 and PC2 together explain only about 41% of the total variance and are constructed without using class label information. Nevertheless, a slight shift in the distribution can be observed along the first principal component, with customers who leave the company tending to be more concentrated at higher values of PC1. Given that PC1 is mainly associated with *Age* and *Balance*, this suggests that churn is more frequent among older customers with higher account balances, which is consistent with the exploratory data analysis.

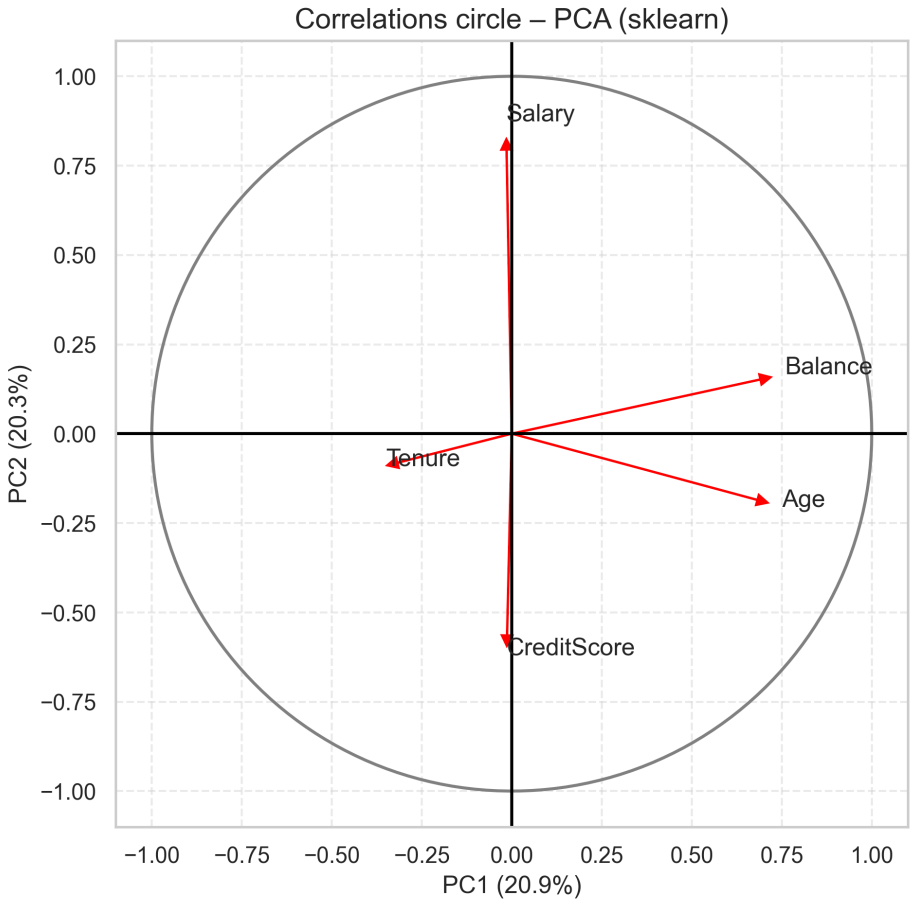


Figure 6: Correlation circle for the first two components

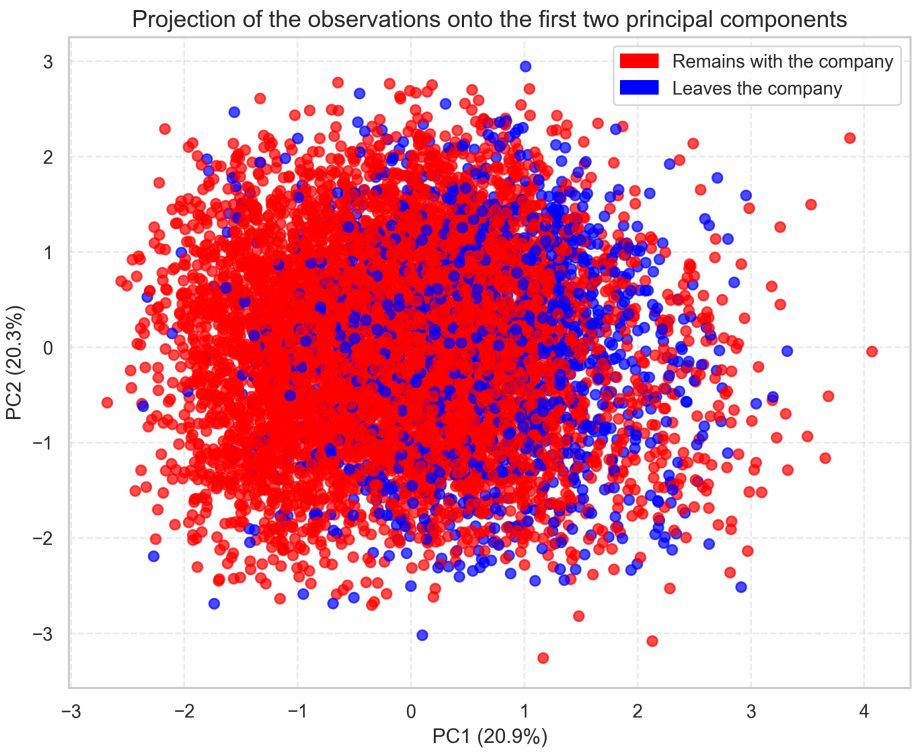


Figure 7: Projection of the observations